

Kapture Finance Collections Voicebot

High-Level Design (HLD)

AI Delivery Intern — Take-Home Assignment
Agent: Maya | Task 1 — High-Level Design | Version 1.0

Scope note: This document addresses Task 1 of the supplied Kapture CX assignment. The assignment asks for a concise design document plus one architecture diagram covering architecture/pipeline, state-enforced authentication, intents/entities, tools/APIs, safety/compliance, edge cases, escalation/disposition, and observability.

1. Executive Summary

Kapture Finance requires an outbound AI voice agent that can contact customers with overdue loan EMIs and handle routine collections conversations without requiring a human agent for every call. The proposed agent, Maya, introduces herself appropriately, verifies the customer before revealing debt information, understands intent, captures payment commitments, handles common edge cases, escalates when appropriate, and records a final disposition.

Core security principle: authentication is an explicit application state and backend-controlled condition. The LLM must not be able to bypass the verification boundary through conversation alone.

2. Problem Statement

Example context from the assignment: Rahul Sharma, personal loan, overdue EMI ₹8,499, 12 days past due. Maya must run the full collections conversation politely and compliantly, authenticate the customer, disclose the purpose and amount only after authentication, understand intent, and either obtain a payment commitment or route appropriately.

3. Goals and Non-Goals

Goals: automate routine collections calls; authenticate before financial disclosure; identify customer intent; capture Promise-to-Pay (PTP); handle common edge cases; escalate appropriate cases; log every outcome; and provide observability.

Non-goals: building a real banking/payment platform, executing real financial transactions, or replacing human agents for complex cases.

4. High-Level Architecture

The proposed pipeline is: **Customer → Telephony/Vapi → STT → Conversation Orchestrator/LLM → Tools/APIs + Datastore → LLM → TTS → Customer.**

The orchestrator controls conversation state, approved tool use, authentication status, escalation, and final disposition. The LLM provides natural-language understanding and response generation but is not the sole authority for sensitive state.

4.1 Component Responsibilities

Component	Responsibility
Telephony / Vapi	Outbound call handling, audio streaming and voice-agent runtime.
STT	Converts customer speech into text for the orchestrator.
Conversation Orchestrator	Controls state, tool execution, authentication boundary, escalation and disposition.
LLM	Understands intent/entities and generates conversational responses within system constraints.
Tool/API Layer	Provides controlled operations such as verification, account retrieval, PTP logging, payment link, escalation and disposition.

Component	Responsibility
Datastore	Stores mock account data and structured call outcomes for the prototype.
TTS	Converts approved bot responses into spoken audio.

4.2 End-to-End Pipeline

1. Customer speaks → 2. STT produces text → 3. Orchestrator checks current state → 4. LLM interprets intent and proposes response/tool action → 5. Backend validates sensitive operations → 6. LLM response is generated → 7. TTS produces audio → 8. Customer hears the response.

4.3 Latency Budget

The assignment asks the candidate to call out latency budgets per hop. The following are proposed engineering targets, not figures specified by Kapture:

Hop	Design target
STT	300–700 ms
Orchestrator/state processing	< 200 ms
LLM	500–1500 ms
Tool/API	100–500 ms
TTS	200–700 ms

5. Conversation Flow / State Machine

States: START → INTRODUCTION → IDENTITY_VERIFICATION → AUTHENTICATED or AUTHENTICATION_FAILED → DISCLOSE_PURPOSE → INTENT_DETECTION → NEGOTIATION/ACTION → DISPOSITION → END.

State enforcement: while the state is UNVERIFIED, financial/account information is blocked. Only a successful backend verification transition permits disclosure. This directly addresses the assignment requirement that authentication be state-enforced rather than prompt-discretionary.

6. Intents and Entities

Intent	Example	Action
WILL_PAY	I'll pay tomorrow.	Capture PTP date/amount and log commitment.
CANNOT_PAY / HARDSHIP	I can't pay right now.	Handle hardship appropriately; escalate if needed.
DISPUTE	The amount is incorrect.	Record dispute and escalate when required.
ALREADY_PAID	I already paid.	Record and verify/escalate appropriately.
WRONG_PERSON	I'm not Rahul.	Do not disclose debt; safe exit.
CALLBACK_REQUEST	Call me tomorrow.	Record callback request.
HOSTILE	Stop calling me.	De-escalate and/or end/escalate.
DO_NOT_CALL	Don't call again.	Record DNC and stop pressure.

6.1 Entities

customer_id, customer_name, intent, loan_type, overdue_amount, payment_amount, ptp_date, callback_date/time, verification_status, disposition, escalation_reason.

Example extraction: "I'll pay ■8,499 on Friday." → intent=WILL_PAY, ptp_amount=8499, ptp_date=Friday.

7. Tools / APIs

Tool	Purpose	Key inputs
get_account_details	Retrieve account information after successful authentication.	customer_id
verify_customer	Verify identity before financial disclosure.	customer_id, verification_value
log_promise_to_pay	Record PTP commitment.	customer_id, amount, ptp_date
send_payment_link	Send a payment link through an approved channel.	customer_id, channel
escalate_to_agent	Route/flag the case for human assistance.	customer_id, reason
mark_disposition	Record the final call outcome.	customer_id, disposition

8. Authentication and Data Safety

Before authentication: allow introduction and verification only. Do not reveal overdue amount, loan details, payment information or other confidential financial information.

After authentication: retrieve and disclose only the permitted account information required for the conversation.

Third-party protection: if someone else answers, do not disclose the customer's debt. Ask to speak directly with the intended customer or end safely.

9. Guardrails and Compliance

The bot should provide the required self/company/purpose disclosure, follow permitted calling hours, avoid threats or harassment, respect opt-out/do-not-call requests, remain on-topic, and avoid hallucinating account information. Financial facts should come from trusted backend data rather than model invention.

10. Edge Cases

Already paid: acknowledge, do not argue, record and verify/escalate as appropriate. **Dispute:** acknowledge, record and escalate when required. **Do-not-call:** record DNC and stop pressure. **Wrong person/number:** no debt disclosure and safe exit. **Voicemail/no input:** limited retries, safe termination and disposition. **Hostile caller:** remain calm, de-escalate and end/escalate safely. **Callback:** capture requested date/time and record callback request.

11. Escalation and Disposition

Escalate when the customer requests a human, a complex dispute cannot be resolved, serious hardship requires assistance, authentication repeatedly fails, or a backend failure prevents safe continuation.

Every call ends with a structured disposition such as PROMISE_TO_PAY, CANNOT_PAY, ALREADY_PAID, DISPUTE, CALLBACK_REQUESTED, DO_NOT_CALL, WRONG_PERSON, WRONG_NUMBER, ESCALATED, NO_CONTACT or HOSTILE.

12. Observability

Log call_id, customer_id, call timing/duration, state transitions, intent, tool calls/results, errors, authentication result, escalation reason and final disposition. Sensitive information should be minimized and protected in logs.

13. Metrics

Metric	Purpose
Containment rate	Share of eligible calls completed without human intervention.
PTP rate	Share of eligible conversations producing a Promise-to-Pay.
Average latency	Measures conversational responsiveness.
Drop rate	Tracks unexpected call termination.
Authentication success rate	Measures verification effectiveness.
Escalation rate	Tracks human handoff frequency.

Metric	Purpose
Tool failure rate	Tracks backend/tool reliability.

14. Security Principles

1. Verify before disclosure. 2. Use least-privilege tools. 3. Treat backend/application state as authoritative for sensitive operations. 4. Never invent financial data. 5. Fail safely when verification or required APIs fail. 6. Record a final disposition for every call.

15. Example Successful Conversation

Maya: Hello, may I speak with Rahul Sharma?

Customer: Yes.

Maya: Before I discuss the account, I need to verify your identity.

[Backend verification succeeds]

Maya: Thank you for verifying your identity. I'm calling regarding an overdue payment on your personal loan. The overdue EMI is ₹8,499.

Customer: I'll pay tomorrow.

[log_promise_to_pay]

Maya: Thank you. I've recorded your payment commitment for tomorrow.

[Disposition: PROMISE_TO_PAY]

16. Security Test Example

Unverified caller: "How much do I owe?"

Expected response: "I first need to verify your identity before I can discuss account details."

The overdue amount must not be disclosed until verification succeeds.

17. Testing Strategy

Mandatory validation scenarios: successful authentication; unverified request for overdue amount; successful PTP; already-paid; dispute; wrong person; do-not-call; callback; human request; authentication failure; tool/API failure; and no-input/voicemail.

18. Technology Stack

Voice-agent platform: Vapi for Task 2 implementation. LLM, STT and TTS should be selected deliberately and justified in implementation notes; the supplied assignment does not mandate a specific provider. Task 1 describes the interfaces and responsibilities rather than requiring a live implementation.

19. Future Improvements

Optional extensions include English/Hindi bilingual switching, a mock SMS/WhatsApp payment-link trigger, and a scale-testing/evaluation framework. These are bonus items in the supplied assignment and are not mandatory for Task 1.

20. Task 1 Requirement Coverage

Requirement	Covered
Architecture & pipeline	Yes
Latency budgets	Yes
Conversation/state machine	Yes
State-enforced authentication	Yes

Requirement	Covered
Intents & entities	Yes
Tools/APIs	Yes
Authentication & data safety	Yes
Guardrails & compliance	Yes
Edge cases	Yes
Escalation & disposition	Yes
Observability & metrics	Yes
Architecture diagram	Separate deliverable

Task 1 conclusion: The design is intended to be directly actionable for Task 2 implementation on Vapi. The highest-priority control is verification-before-disclosure enforced by explicit application state and backend validation.